```
var crypto = new BcTlsCrypto(new SecureRandom());
var server = new PSKTlsServer(crypto,
        new DeviceIdentityManager(deviceDb, encryptionKey)) {
```

You can delete the old `getIdentityManager()` method too because it is unused now. You also need to adjust the `PskClient` implementation to send the correct device ID during the handshake. If you recall from chapter 12, we used an SHA-512 hash of the PSK as the ID there, but the device database uses the ID `"test"` instead. Open PskClient.java and change the `pskId` variable at the top of the `main()` method to use the UTF-8 bytes of the correct device ID:

```
var pskId = "test".getBytes(UTF_8);
```

If you now run the `PskServer` and then the `PskClient` it will still work correctly, but now it is using the encrypted PSK loaded from the device database.

EXPOSING THE DEVICE IDENTITY TO THE API

Although you are now authenticating the device based on a PSK attached to its device profile, that device profile is not exposed to the API after the handshake completes. Bouncy Castle doesn't provide a public method to get the PSK identity associated with a connection, but it is easy to expose this yourself by adding a new method to the `PSKTlsServer`, as shown in listing 13.6. A protected variable